

Lab: Extreme Events

Modeling the Unusual

- For a given sample, the maximum and minimum are fixed
- But if we have a large number of samples, we can study the distribution/density of those max/min draws
- Imagine:
 - The biggest storm in a year
 - The worst loss for the market in a month
 - The biggest earthquake in a decade
 - The longest time between goals in a game
- This is how we understand the distribution/density of extreme events
- The k -th order statistic is the k -th largest value in a sample of n from some distribution
- The minimum is the first order statistic and the maximum is the n -th order statistic

Example: Drawing from a Gaussian

Below, we find the minimum of a sample of size n drawn from a normal distribution, and then repeat this b times, to get a distribution of the minimum with 10000 counts in it. This is the distribution of the 1st order statistic, shown below in blue.

```
In [3]: #!/pip install pyarrow
import pyarrow
import fastparquet
```

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

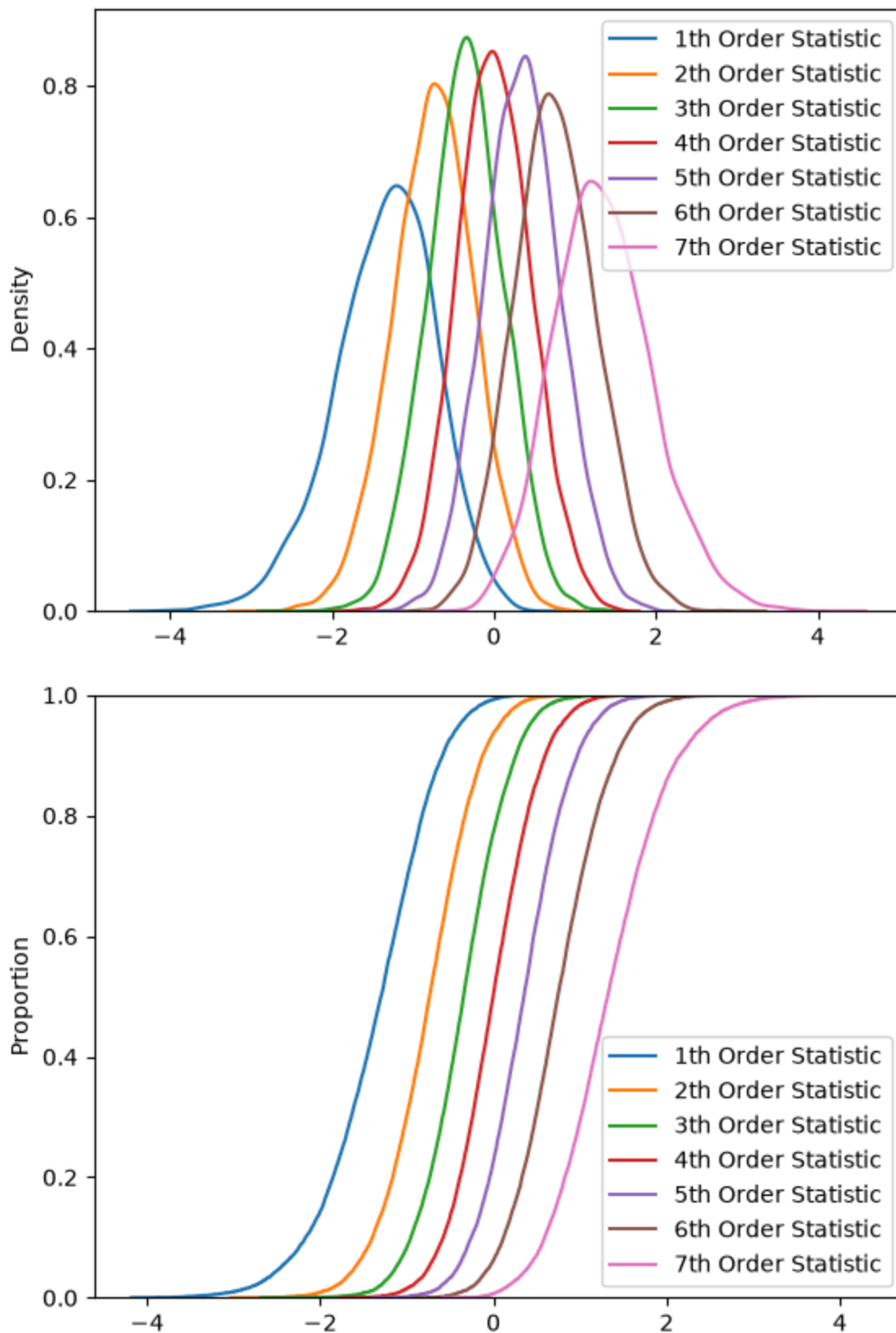
rng = np.random.default_rng( seed = 100 ) # Create a random number stream

n = 7 # Sample Size
b = 10_000 # Number of Simulations

sample = rng.standard_normal( size = (n,b) ) # Draw a sample from the standard normal
sorted_sample = np.sort(sample, axis=0) # Sort each column (each simulated sample)

for j in range(n): # For each rank
    sns.kdeplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot the density
plt.legend() # Add legend
plt.show()
```

```
for j in range(n): # For each rank
    sns.ecdfplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Pl
plt.legend() # Add legend
plt.show()
```



Analytic Relationship

Maximum

If $F(x)$ is the underlying true CDF, let's consider the analytical equation for the distribution of the maximum of a sample of size n . (The maximum is its own random variable, with its distribution plotted above. What is this in math?) To build the CDF of the maximum, we ask (as always): "What's the probability the maximum lands at or below x ?" for every x . In other words:

$$F_{max}(x) = P(X_{max} \leq x)$$

This is nothing new, just applying it to the distribution of a specific order statistic, the max.

So what is the **probability that all of the draws are less than or equal to x** , i.e. what is the probability that the maximum is less than or equal to x (the right hand side)?

Because each draw is independent:

$$P(X_1 \leq x \wedge \dots \wedge X_n \leq x) = P(X_1 \leq x) \cdot P(X_2 \leq x) \cdot \dots \cdot P(X_n \leq x)$$

Because we are pulling from the same distribution every time, **each of these probabilities equals our CDF, $F(x)$** :

$$P(X_1 \leq x, \dots, X_n \leq x) = F(x) \cdot F(x) \cdot \dots \cdot F(x)$$

The probability that all of the draws are less than or equal to x is therefore:

$$F_{max}(x) = F(x) \cdot F(x) \cdot \dots \cdot F(x) = F(x)^n$$

What is its density?

Minimum

The distribution of the minimum is somewhat harder, because our CDF is asking $\leq x$, and " $\max \leq x$ " means " $\text{all} \leq x$ " but " $\min \leq x$ " does not mean " $\text{all} \leq x$ ". We can't just substitute in multiple copies of $F(x)$. Instead of looking at the probability that all draws are less than or equal to x , we need to know the probability that at least one is.

We want to ask "is the minimum less than or equal to x ?" which is the opposite of asking "are all of the draws, including the minimum, above x ?" The probability that all observations are above x is $(1 - F(x))^n$. Because this is the opposite of the question we actually want to ask, the complementary probability is what we truly want:

$$F_{min}(x) = 1 - (1 - F(x))^n$$

What is its density?

All of the other order statistics, generally

- If you're curious, the density of the k -th of n order statistics is

$$f_k(x) = \frac{n!}{(k-1)!(n-k)!} F(x)^{k-1} (1-F(x))^{n-k} f(x)$$

- We'll use the results for the minimum and maximum

The Data

We have three datasets:

- Rainfall *totals* for every state-month from 1981 to 2025, with 5 reporting stations per state each month
- Rainfall for every *station-day* over that same 1981-2025 period (same 5 stations per state) — much bigger, and used for a single question near the end of Step One
- Daily and monthly U.S. stock market returns from 1926 to 2026

We want to understand tail behavior, to better reason about risk (engineering, financial, etc.).

```
In [14]: df = pd.read_parquet("/Users/nasrullahsafdari/UVA/SEMESTERS/FALL-2026/DS5030")
df.head()
```

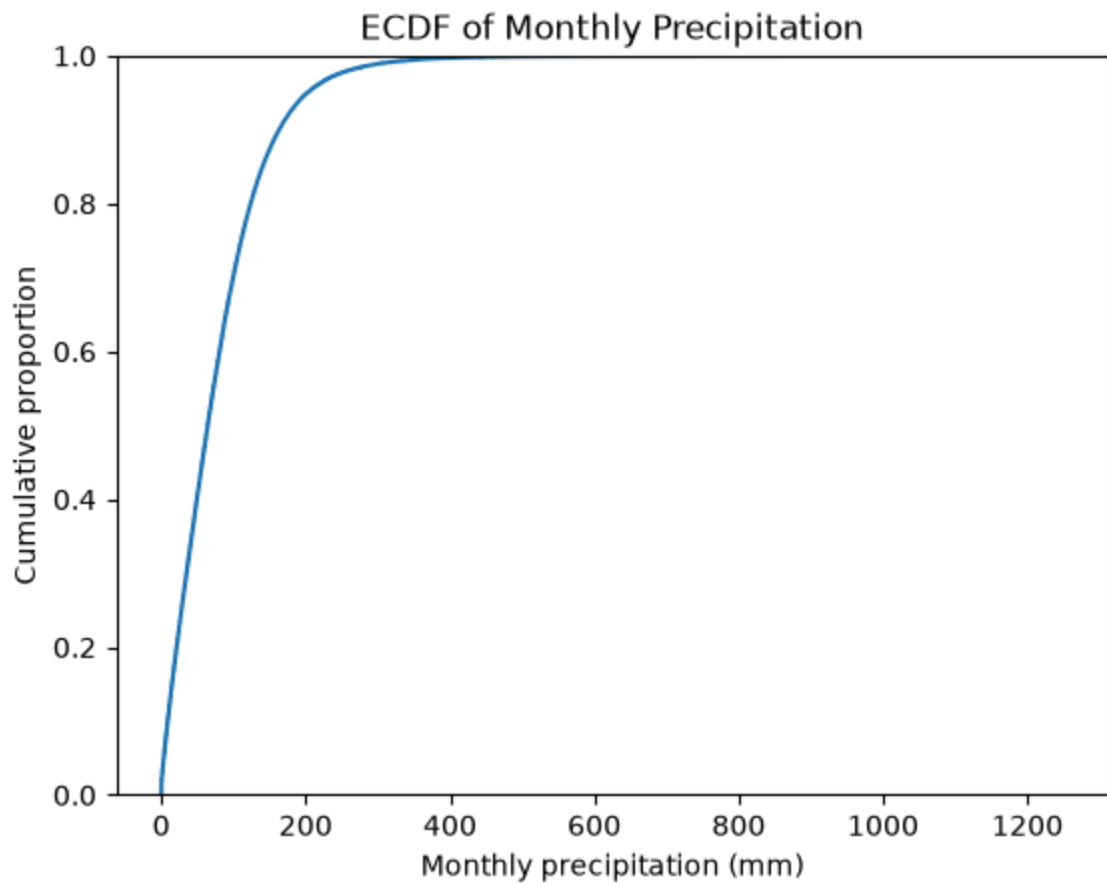
```
Out[14]:
```

	state	state_name	station_rank	station_id	station_name	latitude	longitude
0	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
1	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
2	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
3	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
4	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394

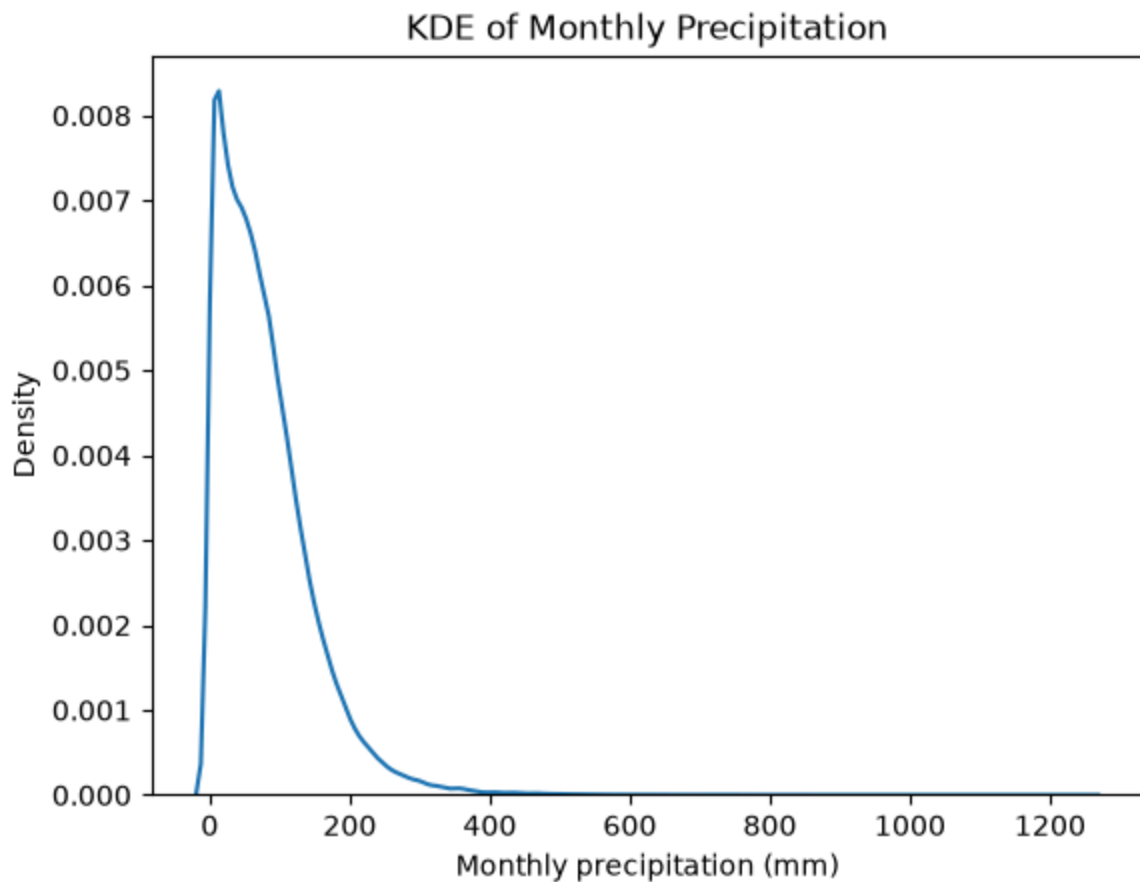
Step One.

- Plot the ECDF and KDE of `prcp_total_mm` with seaborn— the distribution and density of rainfall across the entire U.S.

```
In [6]: sns.ecdfplot(data=df, x="prcp_total_mm")
plt.xlabel("Monthly precipitation (mm)")
plt.ylabel("Cumulative proportion")
plt.title("ECDF of Monthly Precipitation")
plt.show()
```



```
In [7]: sns.kdeplot(data=df, x="prcp_total_mm")
plt.xlabel("Monthly precipitation (mm)")
plt.ylabel("Density")
plt.title("KDE of Monthly Precipitation")
plt.show()
```



2. Now group by year-month and compute the **median** and **maximum** precipitation nationwide. Plot the ECDF and KDE of these two statistics.

```
In [15]: year_month = (
    df.groupby(["year", "month"], as_index=False)
      .agg(
        median_prpc=("prcp_total_mm", "median"),
        max_prpc=("prcp_total_mm", "max")
      )
    )

year_month.head()
```

```
Out[15]:
```

	year	month	median_prpc	max_prpc
0	1981	1	15.100000	194.800003
1	1981	2	69.149998	503.299988
2	1981	3	39.949997	300.000000
3	1981	4	61.899998	229.300003
4	1981	5	82.950001	313.500000

```
In [11]: sns.ecdfplot(
    data=year_month,
```

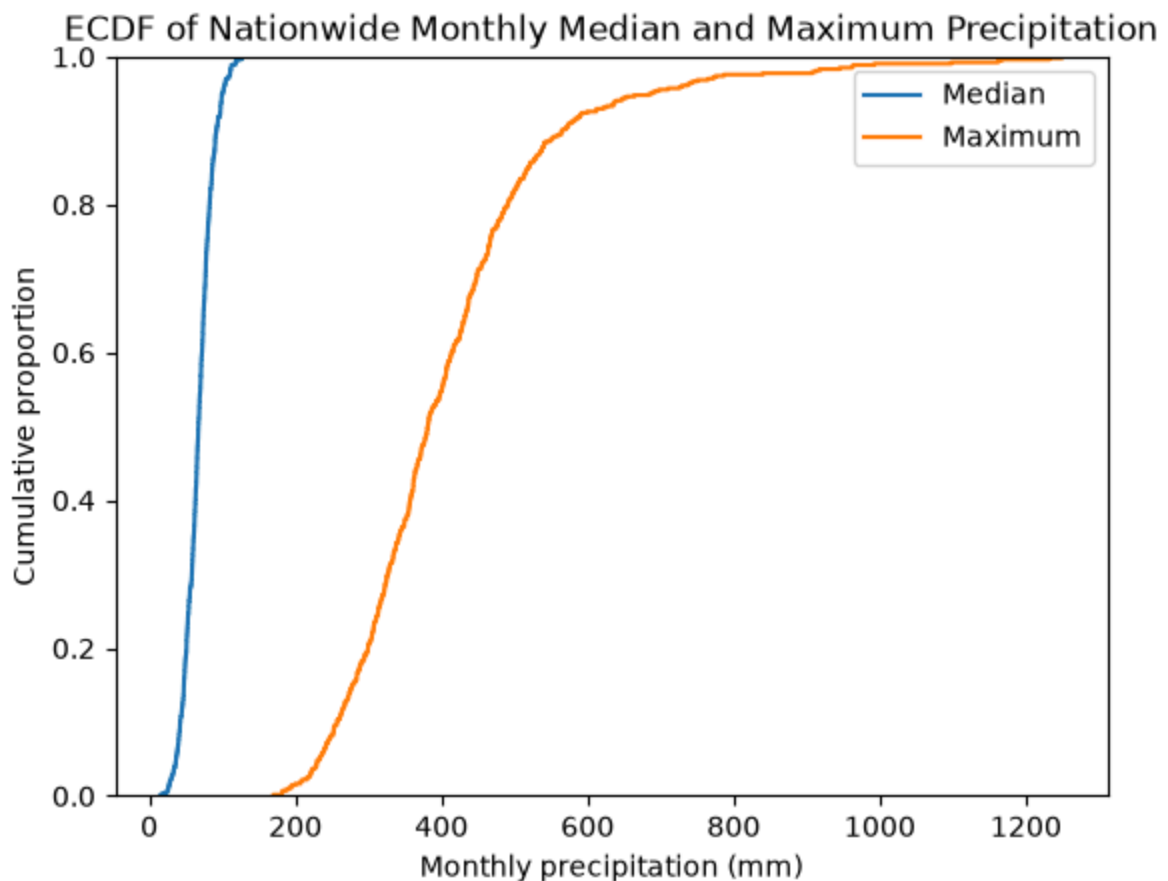
```

    x="median_prcp",
    label="Median"
)

sns.ecdfplot(
    data=year_month,
    x="max_prcp",
    label="Maximum"
)

plt.xlabel("Monthly precipitation (mm)")
plt.ylabel("Cumulative proportion")
plt.title("ECDF of Nationwide Monthly Median and Maximum Precipitation")
plt.legend()
plt.show()

```



```

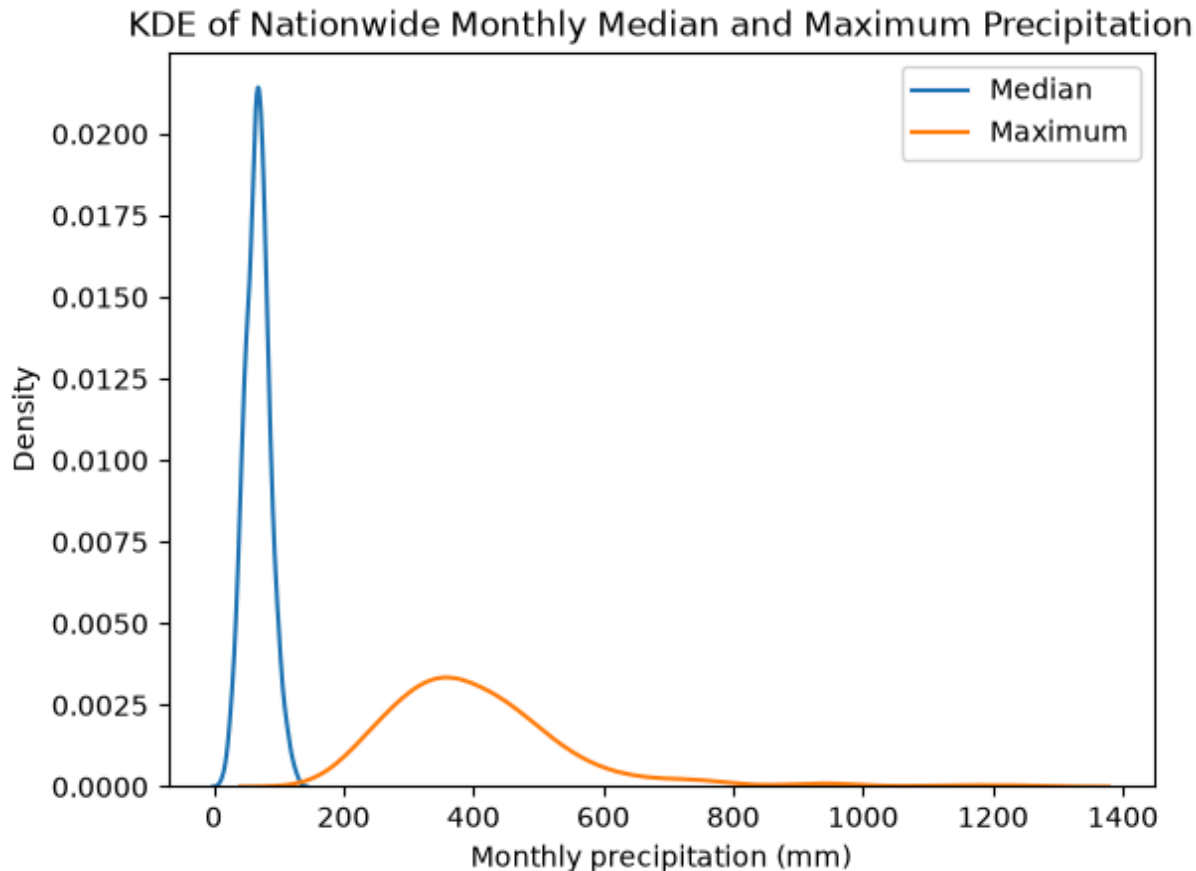
In [12]: sns.kdeplot(
    data=year_month,
    x="median_prcp",
    label="Median"
)

sns.kdeplot(
    data=year_month,
    x="max_prcp",
    label="Maximum"
)

plt.xlabel("Monthly precipitation (mm)")

```

```
plt.ylabel("Density")
plt.title("KDE of Nationwide Monthly Median and Maximum Precipitation")
plt.legend()
plt.show()
```



3. How do your results in part 2 compare to your results in part 1?

Compare and contrast the ECDFs and KDEs from parts 1 and 2. Explain why they are different:

- **Answer:**

In part one ECDF, and KDE checks the distribution of raw observations or we can say the original station level monthly precipitation across all stations, and plotted the ECDF, and KDE of them. As you can see in the graph, the distribution is right skewed, and most of our observations are concentrated at the low/moderate rainfall, and we have small observations with higher rainfall values which resulted to long tail in our density plot.

But in part 2, we grouped the data into month, and year, and we got the nationwide median and maximum. The KDE of median is narrower, and with the higher peak, and its ECDF rises steeply which indicates that monthly median are more stable and concentrated. While KDE of maximum is much more spread out and has lower peak,

and its ECDF moved to the right and spread out and the reason is that for maximum we peaked extreme values of rainfall vlaues from each year-month.

4. Now define a state's rainfall differently: take the **median** of the 5 stations in each month, then find the **maximum** across the state's 12 months in a year — the "wettest month." For each state, what's the expected value of the wettest month? Plot the ECDF of Virginia's annual maxima (the wettest month in each of the 45 years), and overlay the analytical prediction $F_{\max}(x) = \hat{F}(x)^{12}$.

```
In [17]: # One rainfall value per state-month, taking the median of the stations
state_month = (
    df.groupby(["state", "year", "month"], as_index=False)
      .agg(prcp_total_mm=("prcp_total_mm", "median"))
) #TODO median calculation

# Maximum of the 12 state-month observations above, for each year: the wettest
state_year_max = (
    state_month.groupby(["state", "year"], as_index=False)
      .agg(prcp_total_mm=("prcp_total_mm", "max"))
)

exp_wettest = state_year_max.groupby("state", as_index=False).agg(E_wettest_
exp_wettest.loc[exp_wettest["state"] == "VA"]

state = "VA"

levels = (
    state_month.loc[state_month["state"] == state, "prcp_total_mm"]
      .to_numpy()
)

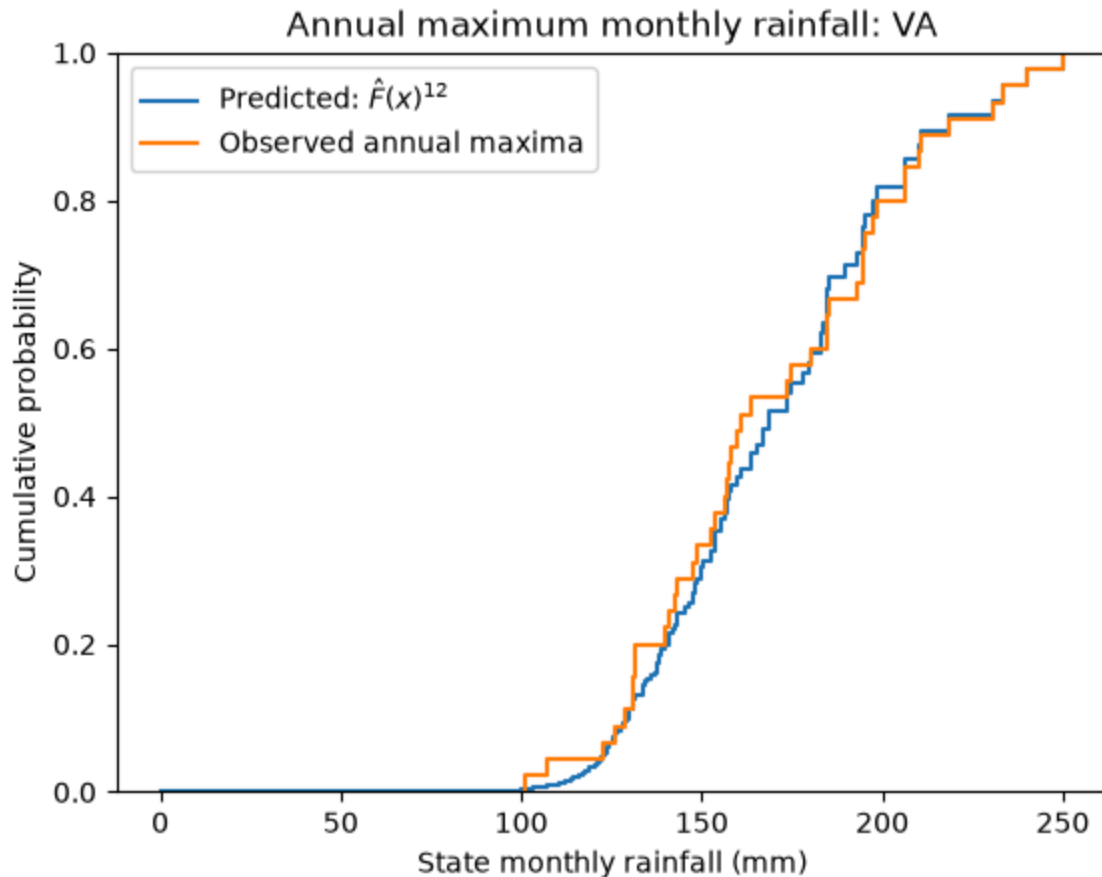
observed_max = (
    state_year_max.loc[state_year_max["state"] == state, "prcp_total_mm"]
      .to_numpy()
)

x = np.sort(np.unique(levels))
F_hat = np.searchsorted(np.sort(levels), x, side="right") / len(levels)

plt.step(
    x,
    F_hat**12,
    where="post",
    label=r"Predicted: $\hat{F}(x)^{12}$"
)

sns.ecdfplot(x=observed_max,
    label="Observed annual maxima")
```

```
plt.xlabel("State monthly rainfall (mm)")
plt.ylabel("Cumulative probability")
plt.title(f"Annual maximum monthly rainfall: {state}")
plt.legend()
plt.show()
```



How close is the fit here? If it differs from question 4's fit, why:

Answer:

As you can see in the graph, both predicted annual maxima and the observed annual maxima curves/graphs follow each other so closely. So the fit is so close, and it provides a better approximation for actual Virginia annual maximum monthly rainfall. There are small differences in some part of the graph, but we can say in general they follow the same pattern. The reason for those small differences is that we assumed that all 12 observations are independent, while in actual rainfall, it is different from season to season and from month to month, which is not really independent.

$$F_{\max}(x) = F(x)^{12}$$

5. **A question for FEMA.** So far you've worked with *monthly* rainfall totals. Switch now to the daily station-level file (`ghcn_prcp_daily.parquet`) — reload `df` from it,

and don't mix it with the monthly data above.

A day with more than 120mm of rain is a serious flood/landslide risk. On any single station-day, that's rare. But a state doesn't get one chance a month — it gets 5 stations $\times$ (days in the month) chances. FEMA doesn't care about the average day, or the single worst day on record; it cares about the probability that *some* station has a disaster day at some point during the month.

For each state-month, estimate

$$P(\text{at least one station-day exceeds 120mm}) = 1 - \hat{F}(120)^n$$

where $\hat{F}$ is the empirical CDF of a single station-day's rainfall for that state and calendar month (pooled across all years), and n is the number of station-days in one occurrence of that month (5 stations $\times$ days in the month). Report the 20 highest-risk state-months.

```
In [18]: # Fresh load -- this question uses only the daily file, not the monthly one
#df_daily = pd.read_parquet("data/ghcn_prdp_daily.parquet")

url = (
    "https://www.dropbox.com/scl/fi/"
    "pjmyop3fqjdy8oabkc4ab/ghcn_prdp_daily.parquet"
    "?rlkey=otxenczb3pxgzz53kc1i5zxww&dl=1"
)
df_daily = pd.read_parquet(url)
df_daily = df_daily.loc[df_daily["observed"]]

threshold = 120 # mm in a single day

def disaster_risk(group):
    F_hat = (group["prcp_mm"] <= threshold).mean()
    n = len(group) / group["year"].nunique() # station-days in one year's c
    return 1 - F_hat**n

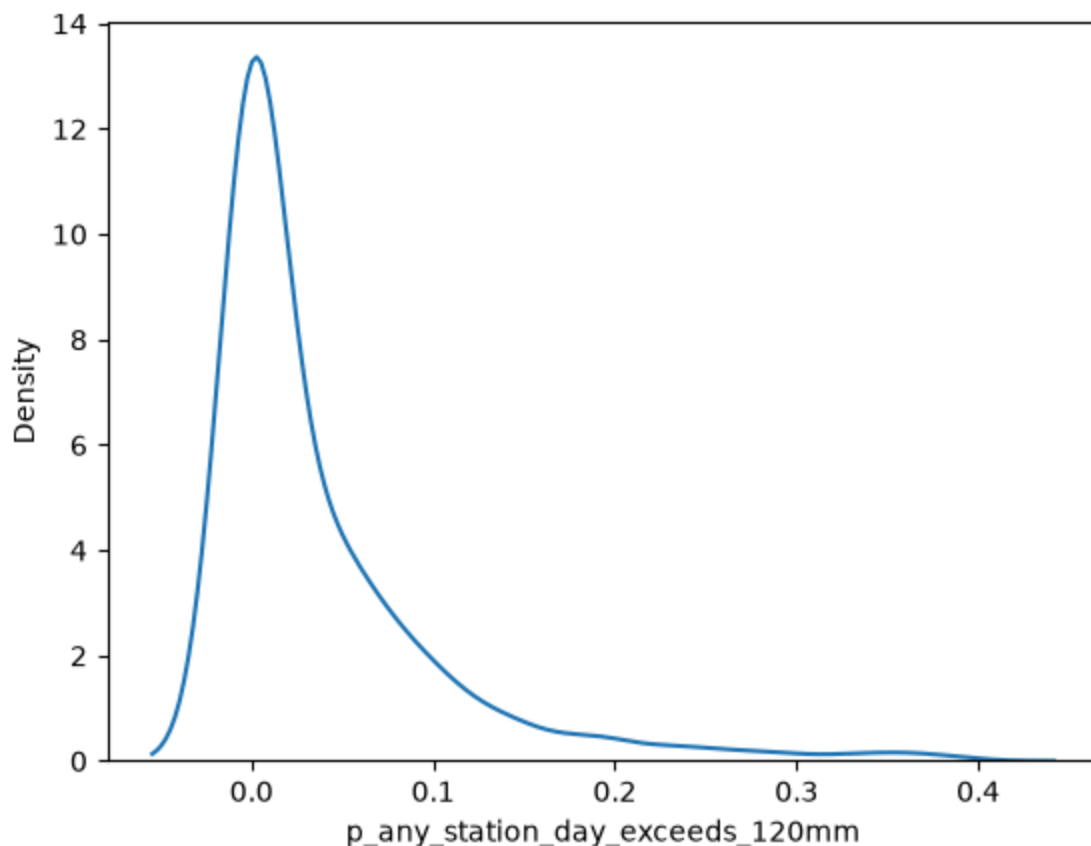
risk = (
    df_daily.groupby(["state", "month"])
    .apply(disaster_risk, include_groups=False)
    .rename("p_any_station_day_exceeds_120mm")
    .reset_index()
    .sort_values("p_any_station_day_exceeds_120mm", ascending=False)
)

sns.kdeplot(risk['p_any_station_day_exceeds_120mm'])

risk.head(20)
```

Out[18]:

	state	month	p_any_station_day_exceeds_120mm
122	HI	3	0.387166
104	FL	9	0.373367
130	HI	11	0.359242
208	LA	5	0.359229
131	HI	12	0.344789
121	HI	2	0.330060
105	FL	10	0.330026
476	SC	9	0.299512
207	LA	4	0.299512
319	NC	8	0.283726
536	VA	9	0.267604
101	FL	6	0.267604
211	LA	8	0.267597
209	LA	6	0.251113
212	LA	9	0.234253
127	HI	8	0.234248
215	LA	12	0.234248
117	GA	10	0.234248
103	FL	8	0.234248
120	HI	1	0.217012



Which states and months face the most risk of a greater-than-120mm day? How high is the probability? Among all state-months, describe the density of high precipitation days:

After seeing the result table, Hawaii in March(3) with the probability **38.7%** face the most risk of a greater-than-129mm of rainfall. I noticed some other states also with the higher risk of Florida and State most risk of a greater-than-129mm of rainfall such as Florida(Sep, 37.3%), and Louisiana(May,35.9%). They listed on multiple time across different months in the table, which shows there are more than one month that these states face the most risk of a greater-than-129mm of rainfall.

The KDE graph also showed that, it is right skewed, and most of the state-months have lower probability of facing risk of a greater-than-129mm of rainfall, and there are fewer with the higher probability which creates the long tail on the right.

Step Two.

Our second data set is daily and monthly U.S. stock market returns. Instead of storms across space (five stations in a state) or across the calendar (twelve months in a year), the "sample" is the set of trading days within a month, and we ask about the best and worst day in that sample.

```
In [30... df = pd.read_parquet("ff_market_daily.parquet")

print(f"{len(df):,} trading days")
print(f"{df['date'].min().date()} through {df['date'].max().date()}")
df.head()
```

26,296 trading days
1926-07-01 through 2026-07-31

```
Out[30]:
```

	date	year	month	month_start	trading_day_of_month	mkt_excess_return_pct
0	1926-07-01	1926	7	1926-07-01	1	0.09
1	1926-07-02	1926	7	1926-07-01	2	0.45
2	1926-07-06	1926	7	1926-07-01	3	0.17
3	1926-07-07	1926	7	1926-07-01	4	0.09
4	1926-07-08	1926	7	1926-07-01	5	0.22

1. Plot the KDE and ECDF of daily market returns.

```
In [25... fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    data=df,
    x="market_return_pct",
    cut=0,
    ax=axes[0],
)
axes[0].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[0].set(
    title="KDE of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Estimated density",
)

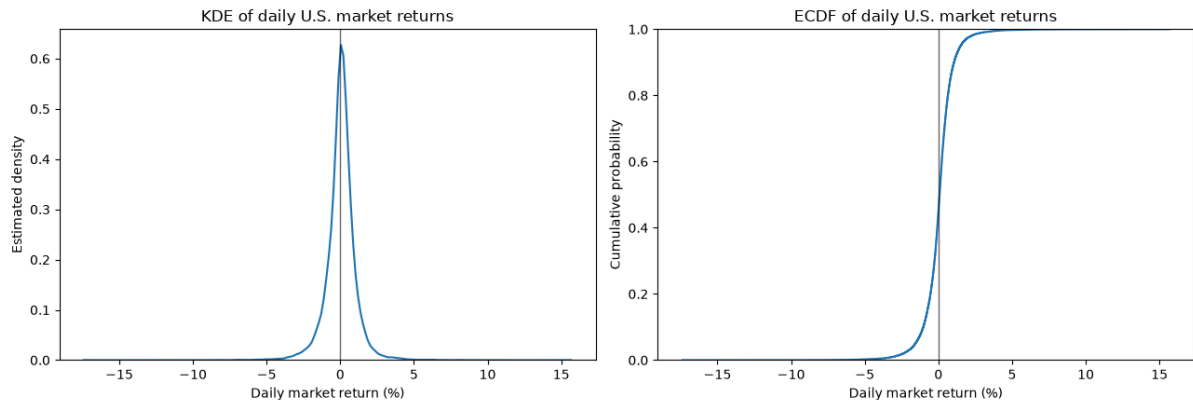
sns.ecdfplot(
    data=df,
    x="market_return_pct",
    ax=axes[1],
)
axes[1].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[1].set(
    title="ECDF of daily U.S. market returns",
    xlabel="Daily market return (%)",
```

```

    ylabel="Cumulative probability",
)

plt.tight_layout()
plt.show()

```



- For each calendar month, compute the **best** (max) and **worst** (min) daily return — the first and n -th order statistics of that month's trading days. Plot the KDE of the monthly best and worst returns, overlaid.

```

In [26... monthly = (
    df.groupby(["year", "month"], as_index=False)
        .agg(
            month_start=("month_start", "first"),
            trading_days=("date", "size"),
            worst_return_pct=("market_return_pct", "min"),
            best_return_pct=("market_return_pct", "max"),
            median_return_pct=("market_return_pct", "median"),
        )
)

# Put losses and gains on the same positive-magnitude scale.
monthly["worst_loss_pct"] = -monthly["worst_return_pct"]

print(f"{len(monthly):,} calendar months")
print(
    f"Trading days per month: {monthly['trading_days'].min()} "
    f"to {monthly['trading_days'].max()}"
)
monthly.head()

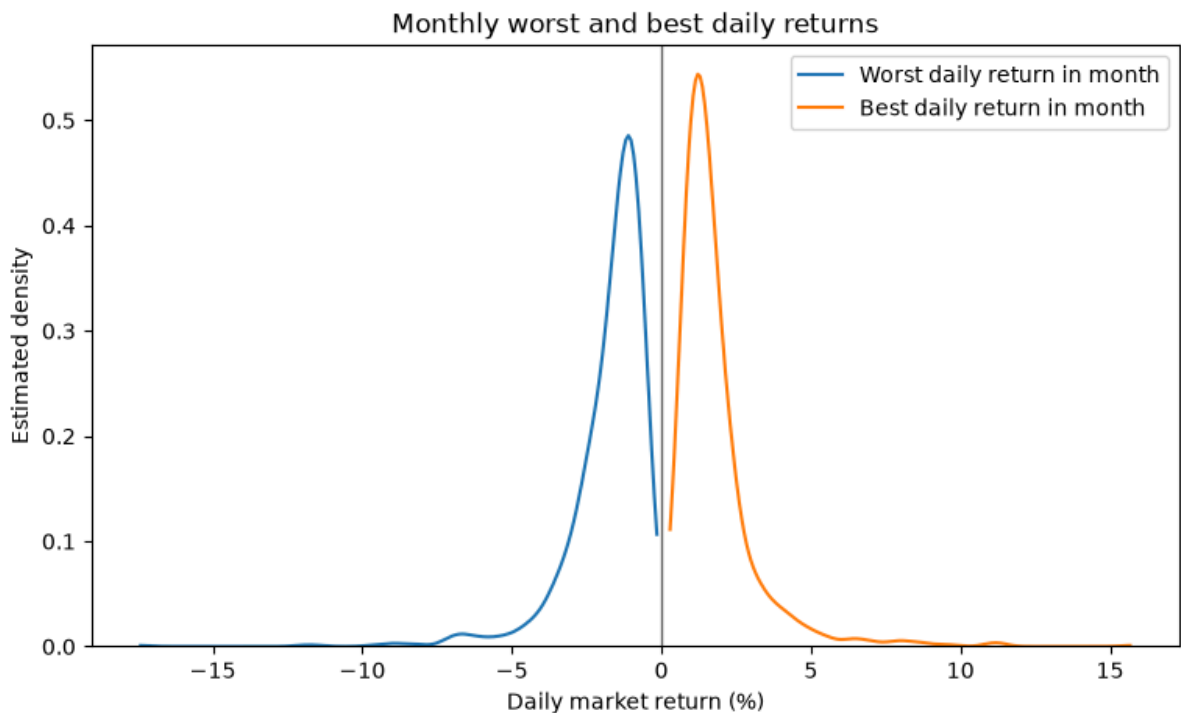
```

1,201 calendar months
 Trading days per month: 15 to 27

Out[26]:

	year	month	month_start	trading_days	worst_return_pct	best_return_pct	mea
0	1926	7	1926-07-01	25	-0.72	1.10	
1	1926	8	1926-08-01	26	-1.39	0.85	
2	1926	9	1926-09-01	24	-0.98	0.91	
3	1926	10	1926-10-01	25	-1.81	1.53	
4	1926	11	1926-11-01	24	-0.62	0.67	

```
In [27... plt.figure(figsize=(9, 5))
sns.kdeplot(
    x=monthly["worst_return_pct"],
    label="Worst daily return in month",
    cut=0,
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best daily return in month",
    cut=0,
)
plt.axvline(0, color="black", linewidth=1, alpha=0.6)
plt.xlabel("Daily market return (%)")
plt.ylabel("Estimated density")
plt.title("Monthly worst and best daily returns")
plt.legend()
plt.show()
```



```
In [31... # more comparing both

# Overall daily returns
print("Overall daily returns:")
```

```
print(df["market_return_pct"].describe())

# Monthly best and worst daily returns
print("\nMonthly best and worst days:")
print(
    monthly[["worst_return_pct", "best_return_pct"]].describe()
)
```

Overall daily returns:

```
count    26296.000000
mean      0.043005
std       1.077241
min      -17.410000
25%      -0.390000
50%       0.080000
75%       0.510000
max       15.670000
Name: market_return_pct, dtype: float64
```

Monthly best and worst days:

	worst_return_pct	best_return_pct
count	1201.000000	1201.000000
mean	-1.777868	1.807460
std	1.397024	1.390837
min	-17.410000	0.290000
25%	-2.210000	1.040000
50%	-1.410000	1.440000
75%	-0.940000	2.040000
max	-0.150000	15.670000

Describe the symmetry (or lack of it) between the best- and worst-day densities. Compare them to the overall populations of daily returns. Use code as appropriate to make these comparisons, and then write out your answer in a markdown cell.

Answer: The graph for daily market return showed that most of the returns are concentrated around zero (median 0.08%), but for the best and worst day of each month, our distribution is a little bit away from zero. As you can see in the above cell, the median of best day return is 1.44% and the median of worst day is -1.41%, so we can say that both distributions are almost symmetric around zero, although, we have a longer tail on the worst day distribution (-17.41%).

So, we are representing the extreme return in the monthly best and worst day return than the daily one.

In []:

3. Losses and gains are on opposite sides of zero, which makes them hard to compare directly. Put them on the same scale by looking at the *magnitude* of the

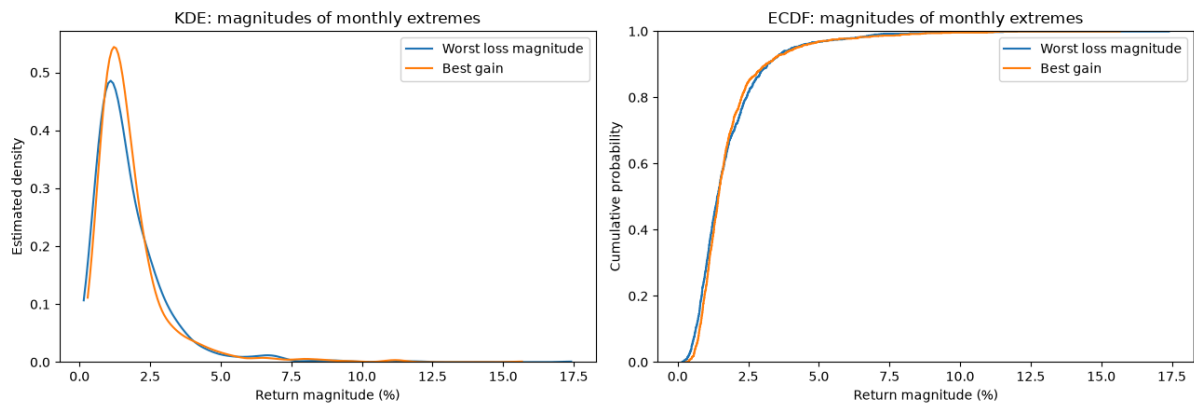
worst loss (`-worst_return_pct`) against the magnitude of the best gain (`best_return_pct`). Plot both the KDE and ECDF.

```
In [32... fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    cut=0,
    ax=axes[0],
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    cut=0,
    ax=axes[0],
)
axes[0].set(
    title="KDE: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Estimated density",
)
axes[0].legend()

sns.ecdfplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    ax=axes[1],
)
sns.ecdfplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    ax=axes[1],
)
axes[1].set(
    title="ECDF: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Cumulative probability",
)
axes[1].legend()

plt.tight_layout()
plt.show()
```



Are large losses more extreme than large gains, or about the same:

Answer:

Now, we can compare them better since we put the magnitude of best and worst monthly return. As you can see in the KDE graph, best and worst loss magnitude overlaps a lot, and most of their density is concentrated between 1-2%, they both have tails on the right side. So, from this graph we can say that in a month the magnitude of gain and loss are almost the same.

It is also the same in the ECDF graph, both curves overlap mostly, so their distributions are almost the same.

4. Compute the median, 75th, 90th, 95th, and 99th percentile of both magnitudes from question 3. In what fraction of months does the worst loss exceed the best gain in magnitude?

```
In [33... tail_summary = pd.DataFrame(
    {
        "Worst loss magnitude": monthly["worst_loss_pct"],
        "Best gain": monthly["best_return_pct"],
    }
).quantile([0.50, 0.75, 0.90, 0.95, 0.99]).T

tail_summary.columns = ["median", "75%", "90%", "95%", "99%"]

print(
    "Fraction of months in which the worst loss exceeded the best gain: "
    f"{(monthly['worst_loss_pct'] > monthly['best_return_pct']).mean():.1%}"
)
tail_summary.round(2)
```

Fraction of months in which the worst loss exceeded the best gain: 44.8%

Out [33]:

	median	75%	90%	95%	99%
Worst loss magnitude	1.41	2.21	3.18	4.06	6.97
Best gain	1.44	2.04	3.12	4.21	7.93

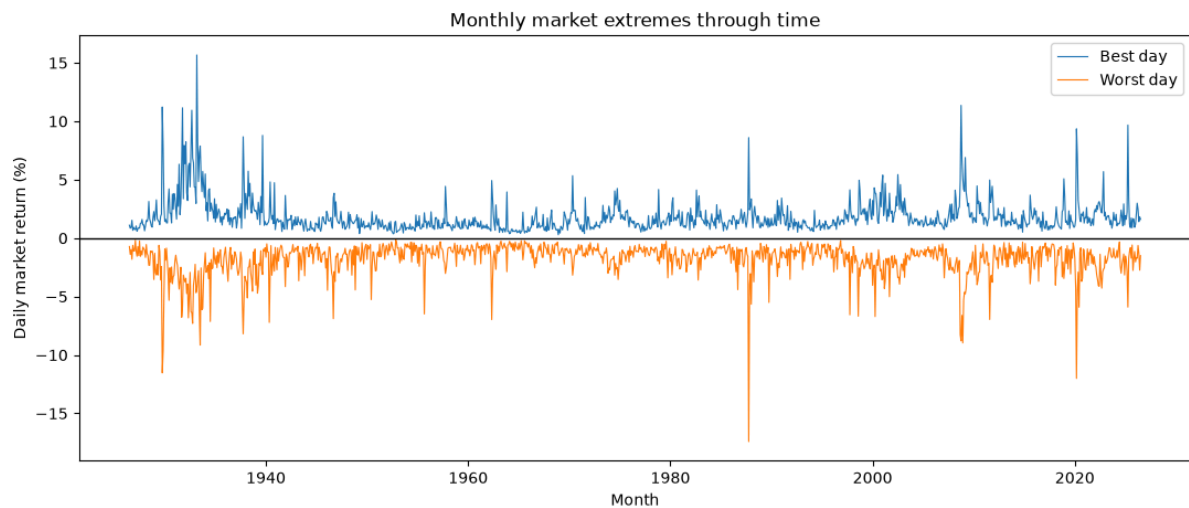
What does this tail comparison say about the risk of losses vs. gains:

Answer:

Overall, we can say that the magnitude of losses and best return are almost same, and as you can see their median and lower percentile are pretty close to each other. But, in higher percentile, 99% for instance, their difference is a lot and monthly best gain is actually larger than monthly worst losses magnitude. Also, we got that in 44.8% of the months worst losses are the larger than best returns. It doesn't mean that that we have more extremes in large losses compare to best return, we can generally say they have similar behavior.

5. Plot the monthly best and worst daily returns over time. Do the extremes look stable across this ~100-year history, or are there periods where they're much larger? Can you identify any famous market events?

```
In [34... plt.figure(figsize=(13, 5))
plt.plot(
    monthly["month_start"],
    monthly["best_return_pct"],
    linewidth=0.8,
    label="Best day",
)
plt.plot(
    monthly["month_start"],
    monthly["worst_return_pct"],
    linewidth=0.8,
    label="Worst day",
)
plt.axhline(0, color="black", linewidth=1)
plt.xlabel("Month")
plt.ylabel("Daily market return (%)")
plt.title("Monthly market extremes through time")
plt.legend()
plt.show()
```



Do upside and downside risk tend to go together (volatility), or does the market exhibit distinct periods in which the good shocks are concentrated and the bad shocks are concentrated? What implication does this have for investing?

Answer: Based on the graph above, the upside and downside moves together (volatility), which means if there is large movement, we have it on both best and worst day return in the volatile periods. As you can see, we have large spikes (movement) in around 1930, 1988, 2006-2008, 2020 on both returns. So for investing, we can say that we have greater uncertainty in both directions, since we have high volatility in usually both large losses and best returns.

Wrap-up

This week has focused on kernel density estimation, particularly estimating the PDF from data. This lab takes that a step further: We can estimate and quantify the densities and distributions of random variables that build off of observed phenomena. What is the difference between the ECDF and KDE? Explain how these estimations today give us different information than an ECDF or KDE plot of the underlying data, but quantify information about the behavior of a different random variable.

Answer:

The difference between ECDF and KDE is that **ECDF** shows the 'cumulative distribution of the observed data', but the **KDE** is "the smoothed estimation of density or PDF of underlying data".

In this week, we didn't work on the distribution of the original variable, but we transformed it and created new random variables from them such as max, median of rainfall of months (12), or best/worst market return, and looked at their

distributions. It helps us to get information about the probability of extreme events, as it was explained in the beginning of the lab, which helps us to know about the uncertainty or risk.

AI Use:

I used ChatGPT to help debug code, understand and explain parts of the code, and clarify statistical concepts used in the lab. I also used Google to look up the definition of kernel density estimation (KDE).

OpenAI. (2026). ChatGPT (GPT-5.6 Sol) [Large language model]. ChatGPT